

GemChat

Opis Struktury Projektu i Architektury — Wersja Rozszerzona

1. O Projekcie

GemChat to zaawansowana aplikacja mobilna na platformę Android, działająca jako prywatny, inteligentny asystent wykorzystujący modele językowe Google Gemini. Projekt wspiera multimodalność, co oznacza, że przetwarza zarówno tekst, jak i obrazy dostarczane przez użytkownika. Kluczowym filarem aplikacji jest podejście *Privacy-by-Design* – cała historia czatów, załączniki oraz metadane konwersacji są zapisywane wyłącznie lokalnie w piaskownicy (sandboxie) aplikacji, eliminując konieczność synchronizacji z zewnętrznymi chmurami bazodanowymi.

2. Szczegółowy Opis Warstwy Repozytorium (Data Repository)

Warstwa repozytorium w projekcie GemChat implementuje wzorzec projektowy *Repository Pattern*. Stanowi ona centralny punkt dostępu do danych, izolujący logikę biznesową od fizycznych źródeł danych (bazy danych i API sieciowego). Taka separacja gwarantuje, że zmiana sposobu przechowywania danych nie wpływa na działanie interfejsu użytkownika.

ChatRepository.kt (Zarządzanie lokalną bazą danych)

Odpowiada za pośredniczenie pomiędzy aplikacją a lokalną bazą danych SQLite zarządzaną przez bibliotekę Room. Udostępnia metody synchroniczne i asynchroniczne (wykorzystujące Kotlin Coroutines) do zapisu i odczytu:

- **Strumieniowanie danych (Flow):** Zwraca dane w postaci reaktywnych strumieni `Flow<List<Conversation>>` lub `Flow<List<Message>>`. Dzięki temu każda zmiana w bazie danych natychmiast automatycznie odświeża interfejs użytkownika bez konieczności ponownego wysyłania zapytań.
- **Operacje CRUD:** Udostępnia funkcje zawieszalne (`suspend fun`) do dodawania nowych wiadomości, tworzenia sesji konwersacji oraz usuwania wątków.

GeminiRepository.kt (Komunikacja z zewnętrznym AI)

Odpowiada w całości za asynchroniczną komunikację sieciową z oficjalnym API Google AI Studio. Warstwa ta zarządza:

- **Konfiguracją modelu:** Definiuje parametry zapytania (np. wybór wersji modelu Gemini 1.5 Flash lub Pro, limity tokenów).
- **Przetwarzaniem multimodalnym:** Przyjmuje zapytania tekstowe oraz konwertuje pliki graficzne przekazywane z interfejsu na odpowiedni format binarny (Base64/Bitmap) akceptowany przez endpoint API sieciowego.

3. Szczegółowy Opis Warstwy ViewModel

Klasy ViewModel w architekturze MVVM odpowiadają za przygotowanie, transformację i utrzymanie stanu interfejsu użytkownika (UI State). Są one odporne na zmiany konfiguracji urządzenia (np. obracanie ekranu) i żyją dłużej niż same widoki Compose.

Zarządzanie Stanem (UI State & StateFlow)

ViewModels w GemChat nie udostępniają surowych danych. Zamiast tego implementują nowoczesne, reaktywne podejście za pomocą StateFlow lub MutableStateFlow. Reprezentują one stan ekranu jako jeden spójny obiekt (np. ChatUiState), który przechowuje flagi takie jak: czy dane się ładują (isLoading), treść aktualnej listy wiadomości czy ewentualne komunikaty o błędach sieciowych.

Asynchroniczność i Cykl Życia (viewModelScope)

Wszystkie operacje długotrwałe, takie jak zapytania do repozytoriów, uruchamiane są wewnątrz bezpiecznego kontekstu viewModelScope.launch. Zapewnia to automatyczne anulowanie wątków tła i zapytań sieciowych w sytuacji, gdy użytkownik zamknie dany ekran, zapobiegając wyciekom pamięci (memory leaks) i niepotrzebnemu zużyciu baterii.

4. Stos Technologiczny

Technologia	Zastosowanie w projekcie
Kotlin	Główny, nowoczesny język programowania aplikacji

Jetpack Compose	Budowa deklaratywnego interfejsu użytkownika (zamiast XML)
Room Database	Lokalna baza danych (SQLite) do przechowywania historii czatów
Gemini REST API	Integracja z modelem AI (generowanie treści, analiza obrazu)
Coil	Asynchroniczne ładowanie i wyświetlanie obrazów

5. Struktura Katalogów Projektu

```

com.gemchat.app/
├─ GemChatApplication.kt # Inicjalizacja bazy danych i repozytoriów
├─ MainActivity.kt # Punkt startowy, konfiguracja Jetpack Navigation
├─ data/
│   ├─ GemChatDatabase.kt # Abstrakcyjna klasa bazy danych Room
│   ├─ dao/
│   │   └─ ChatDao.kt # Interfejs zapytań SQL SQL (Data Access Object)
│   └─ model/
│       └─ Entities.kt # Klasy danych / tabele (Conversation, Message, Attachment)
├─ repository/
│   ├─ ChatRepository.kt # Warstwa abstrakcji nad lokalną bazą danych
│   └─ GeminiRepository.kt # Warstwa obsługi żądań do Google Gemini API
├─ ui/
├─ chat/
│   ├─ ChatScreen.kt # Kompozycja widoku okna rozmowy
│   └─ ChatViewModel.kt # Logika czatu, wywołania asynchroniczne AI
├─ conversations/
│   ├─ ConversationsScreen.kt # Interfejs listy wątków archiwalnych
│   └─ ConversationsViewModel.kt # Zarządzanie listą sesji użytkownika
├─ settings/
├─ SettingsScreen.kt # Panel preferencji, zmiana motywów graficznych
└─ SettingsViewModel.kt # Modyfikacja ustawień lokalnych aplikacji

```

6. Podsumowanie i Zalety Edukacyjne

Połączenie wzorca **Repository Pattern** oraz dedykowanych klas **ViewModel** pozwala na pełne uniezależnienie komponentów. Interfejs użytkownika (Jetpack Compose) jedynie "obserwuje" stan dostarczany przez ViewModel, który z kolei bezpiecznie pobiera dane z Repozytorium. Architektura

ta doskonale symuluje systemy rozproszone o wysokiej niezawodności i niskim stopniu powiązania komponentów (low coupling).

Wygenerowano na potrzeby weryfikacji i prezentacji projektu "GemChat" (Aplikacje Mobilne 2025).